

Infraestrutura de TI e Computação em Nuvem

Aula 6

Sistemas operacionais: processos, threads, memória, entrada/saída, arquivos e escalonamento

Disciplina	Infraestrutura de TI e Computação em Nuvem
Peso no edital	8 questões nos Conhecimentos Específicos
Banca	FGV - Fundação Getulio Vargas
Foco da aula	Sistemas operacionais: processos, memória, entrada e saída
Estratégia	Fundamentos sem pular base, com aplicação em servidor Java/Quarkus, APIs e ambiente corporativo

Material original, orientado ao edital TJSC 2026 e calibrado para o padrão de questões da FGV. Não reproduz questões reais.

1. Identificação da aula e recorte do edital

A aula pedida é a Aula 6 da trilha nova deste chat: Sistemas operacionais: processos, threads, memória, entrada/saída, arquivos e escalonamento.

No edital de Infraestrutura de TI e Computação em Nuvem do TJSC, esta aula cobre diretamente o item 1.6: Sistemas operacionais: processos, memória, entrada e saída. Ela também conversa, de forma instrumental, com infraestrutura de TI, servidores, armazenamento, virtualização e containers, mas sem deslocar o foco para as aulas seguintes.

A lógica de prova é simples: a FGV pode cobrar o conceito puro, mas tende a preferir cenário técnico. Por exemplo: um processo bloqueado por E/S, uma aplicação Java com várias threads, um servidor com thrashing, um escalonador preemptivo com quantum, ou um sistema de arquivos com metadados e permissões.

O Manual Mestre de Calibração FGV do projeto exige aderência ao edital, explicação progressiva, exemplos específicos da disciplina, questões com distratores plausíveis, gabarito orgânico e comentários completos. Este material segue essa régua.

1.1 O que será coberto

Bloco	Conteúdo	Objetivo de prova
1	Papel do sistema operacional, kernel, modo usuário, modo kernel e chamadas de sistema	Entender o SO como gerente de recursos e intermediário controlado entre aplicação e hardware
2	Processos, estados, PCB, contexto e troca de contexto	Diferenciar programa, processo, processo pronto, em execução e bloqueado
3	Threads, concorrência, paralelismo e sincronização	Distinguir processo de thread e evitar confusão com execução paralela
4	Escalonamento de CPU	Comparar FCFS, SJF, SRTF, Round Robin, prioridade, starvation e aging
5	Gerenciamento de memória	Entender memória virtual, paginação, TLB, page fault, swapping e thrashing
6	Entrada/saída	Diferenciar polling, interrupção, DMA, buffer, cache e spool
7	Sistemas de arquivos	Compreender arquivo, diretório, metadados, permissões, descritor e journaling
8	Questões FGV	Treinar raciocínio comparativo com 20 questões autorais

1.2 Como a FGV vem cobrando esse eixo

Em provas recentes de TI, a FGV não fica apenas na definição de “o que é processo”. Ela já cobrou memória virtual com page-fault frequency e thrashing, threads em Java com Runnable, camadas OSI com segmentação de dados, firewalls em cenário técnico e métodos de entrada/saída como DMA. O nível é de média a alta discriminação: o candidato que sabe o termo, mas não sabe a consequência, erra.

Portanto, nesta aula, cada conceito será explicado por função, limite, erro comum e aplicação prática. A meta não é virar especialista em kernel; é acertar questão objetiva sem cair em troca de termos próximos.

2. Siglas e termos essenciais da aula

Sigla/termo	Expansão	Como memorizar para prova
SO	Sistema Operacional	Software-base que gerencia CPU, memória, E/S, arquivos e usuários.

Sigla/termo	Expansão	Como memorizar para prova
CPU	Central Processing Unit - Unidade Central de Processamento	Executa instruções; o escalonador decide quem usa a CPU.
UCP	Unidade Central de Processamento	Termo em português para CPU.
Kernel	Núcleo do sistema operacional	Parte privilegiada que controla recursos e intermedeia acesso ao hardware.
API	Application Programming Interface	Interface usada pela aplicação para pedir serviços; syscalls são interface controlada com o kernel.
Syscall	System Call - chamada de sistema	Ponte controlada entre modo usuário e modo kernel.
PCB	Process Control Block - Bloco de Controle de Processo	Registro interno do SO com estado, registradores, prioridade, memória e E/S do processo.
PID	Process Identifier	Identificador de processo.
TCB	Thread Control Block	Estrutura de controle de thread.
RAM	Random Access Memory	Memória principal, volátil, onde programas em execução mantêm dados/instruções.
MMU	Memory Management Unit	Hardware que traduz endereços lógicos/virtuais em físicos.
TLB	Translation Lookaside Buffer	Cache de traduções de endereço para acelerar paginação.
E/S	Entrada/Saída	Comunicação entre CPU/memória e dispositivos, como disco, teclado, rede e impressora.
I/O	Input/Output	Entrada/Saída em inglês.
IRQ	Interrupt Request	Sinal de interrupção usado por dispositivo para chamar atenção da CPU.
DMA	Direct Memory Access	Acesso direto à memória por controlador, sem CPU copiando byte a byte.
SSD	Solid State Drive	Armazenamento persistente sem partes mecânicas.
HDD	Hard Disk Drive	Disco magnético tradicional.
FIFO	First In, First Out	Primeiro a entrar, primeiro a sair. Pode aparecer em escalonamento ou substituição de páginas.
FCFS	First-Come, First-Served	Processo atendido na ordem de chegada.
SJF	Shortest Job First	Menor job primeiro; minimiza espera média em cenário ideal, mas exige estimar duração.
SRTF	Shortest Remaining Time First	Versão preemptiva do SJF: menor tempo restante.
RR	Round Robin	Escalonamento por fatias de tempo, típico de sistemas interativos.
POSIX	Portable Operating System Interface	Família de padrões de interface de sistemas tipo Unix.
JVM	Java Virtual Machine	Processo que executa bytecode Java; sua aplicação Quarkus roda dentro dela.
GC	Garbage Collection	Coleta automática de memória em ambientes como Java. Não substitui gerência de memória do SO.

3. O que é um sistema operacional

Sistema operacional é o software que gerencia recursos de hardware e oferece uma interface controlada para programas e usuários. Sem ele, cada aplicação precisaria saber falar diretamente com CPU, memória, disco, placa de rede, teclado, monitor e periféricos. Isso seria inseguro, frágil e impraticável.

Para a prova, pense no SO como um gerente de recursos. Ele decide qual processo usa a CPU, qual área de memória cada processo pode acessar, como uma operação de disco será executada, como arquivos são

organizados e como dispositivos de entrada e saída serão abstraídos.

3.1 Kernel, modo usuário e modo kernel

O **kernel é o núcleo do sistema operacional**. Ele executa com **privilegios elevados e controla recursos sensíveis**. As aplicações comuns executam em modo usuário, com privilégios restritos. Quando uma aplicação precisa de um serviço privilegiado, como abrir arquivo, criar processo, alocar memória ou enviar dados pela rede, ela usa uma chamada de sistema.

Conceito	Ideia central	Exemplo prático
Modo usuário	Execução restrita das aplicações	API Java/Quarkus rodando na JVM, navegador, editor de texto
Modo kernel	Execução privilegiada do núcleo do SO	Gerência de memória, escalonamento, drivers, interrupções
Chamada de sistema	Pedido controlado da aplicação ao kernel	open/read/write, criação de processo, envio por socket
Driver	Software que permite ao SO controlar dispositivo	Driver de disco, placa de rede, GPU, teclado

Pegadinha FGV: aplicação não acessa hardware livremente como se fosse dona da máquina. Ela pede serviços ao sistema operacional. O SO protege recursos, arbitra conflitos e isola processos.

3.2 Analogia com aplicações corporativas

Em uma arquitetura Java/Quarkus, você não deixa o front-end Angular gravar diretamente no banco. Ele chama uma API, que valida regras, autentica, autoriza e persiste. No SO ocorre algo análogo: o programa não deve manipular diretamente a controladora do disco ou a tabela de páginas; ele chama o kernel, que valida e executa a operação conforme permissões e políticas.

4. Processos

Programa é código armazenado. Processo é programa em execução, com estado próprio. Um mesmo programa pode originar vários processos. Abrir dois navegadores ou executar duas instâncias da mesma API pode criar processos distintos, cada um com seu espaço de endereçamento, descritores, contexto e recursos associados.

4.1 O que há dentro de um processo

Um processo não é apenas código. Ele inclui código executável, dados, pilha, heap, registradores, contador de programa, arquivos abertos, informações de segurança e estado de execução. O sistema operacional precisa manter essas informações para pausar, retomar, bloquear, terminar e escalonar processos.

Elemento	Função	Pegadinha comum
Código/texto	Instruções do programa	Não confundir programa no disco com processo em execução.
Heap	Área de alocação dinâmica	Em Java, objetos ficam logicamente no heap da JVM; o SO enxerga páginas de memória do processo.
Pilha/stack	Chamadas de função, variáveis locais e contexto de execução	Cada thread normalmente possui sua própria pilha.
Registradores e PC	Estado da CPU no momento da execução	Precisam ser salvos na troca de contexto.
Descritores/handles	Referências a arquivos, sockets e outros recursos	Arquivo aberto não é o arquivo em si; é uma referência controlada.
PCB	Bloco de Controle de Processo	Não é código do programa; é estrutura de controle mantida pelo SO.

4.2 Estados clássicos de um processo

A visão clássica de concurso usa cinco estados: **novo, pronto, em execução, bloqueado/esperando e terminado**. A FGV pode descrever o estado por cenário, não pelo nome.

Estado	Significado	Exemplo
Novo	Processo está sendo criado	SO cria estruturas de controle para iniciar uma aplicação.
Pronto	Tem tudo para executar, mas aguarda CPU	API pronta para processar, esperando ser escolhida pelo escalonador.
Em execução	Está usando CPU	Thread executa instruções de negócio ou serialização JSON.
Bloqueado/esperando	Aguarda evento externo ou recurso	Processo espera leitura de disco, rede, lock ou entrada de usuário.
Terminado	Finalizou ou foi encerrado	Processo encerrou, falhou ou foi morto pelo SO/administrador.

Ponto de prova: pronto não é bloqueado. O processo pronto espera apenas CPU. O processo bloqueado espera evento ou recurso, como E/S, lock, sinal ou dado de rede.

4.3 Troca de contexto

Troca de contexto é o procedimento pelo qual **o SO salva o estado da entidade em execução e carrega o estado de outra**. É indispensável para multitarefa, mas tem custo: **enquanto troca contexto, a CPU não está executando trabalho útil da aplicação**.

A FGV pode tentar confundir escalonamento com troca de contexto. **Escalonamento é a decisão de quem deve executar**; troca de contexto é a operação de salvar/restaurar contexto para efetivar a troca.

4.4 Processo CPU-bound e I/O-bound

Tipo	Descrição	Exemplo de dev
CPU-bound	Consome muito processador	Compressão, criptografia pesada, processamento de imagem, cálculo em lote.
I/O-bound	Passa muito tempo aguardando E/S	API esperando banco, disco, rede, fila, chamada HTTP externa.

Essa distinção é importante porque algoritmos de escalonamento e desenho de aplicações lidam de forma diferente com tarefas curtas/interativas, longas em background, bloqueantes e intensivas em CPU.

5. Threads

Thread é um fluxo de execução dentro de um processo. Um processo pode ter uma ou várias threads. **Threads de um mesmo processo compartilham recursos como espaço de endereçamento**, heap e arquivos abertos, **mas cada thread possui seu próprio contexto de execução, como registradores, contador de programa e pilha**.

O ponto mais cobrado é a comparação: **processo tem isolamento mais forte; thread é mais leve, mas compartilha memória e aumenta risco de corrida se houver acesso concorrente a dados mutáveis**.

5.1 Processo x thread

Critério	Processo	Thread
Isolamento	Maior: espaço de endereçamento próprio	Menor: compartilha memória do processo
Custo de criação/troca	Geralmente maior	Geralmente menor
Comunicação	IPC costuma ser mais custoso	Compartilha variáveis/heap, mas exige sincronização
Falha	Falha tende a afetar o processo	Falha em thread pode comprometer o processo inteiro

Critério	Processo	Thread
Exemplo	JVM da aplicação Quarkus como processo	Threads atendendo requisições HTTP dentro da JVM

5.2 Concorrência x paralelismo

Concorrência é lidar com várias tarefas em andamento, intercalando ou coordenando sua execução.

Paralelismo é executar **tarefas literalmente ao mesmo tempo em múltiplos núcleos ou processadores**. Em um único núcleo, pode haver concorrência por alternância rápida, mas não paralelismo real.

Conceito	Pergunta mental	Exemplo
Concorrência	Há várias tarefas progredindo de modo intercalado?	Servidor atende várias requisições alternando threads.
Paralelismo	Há execução simultânea física?	Duas threads em núcleos diferentes executando ao mesmo tempo.

5.3 Threads em Java/Quarkus

No seu contexto, uma aplicação Quarkus roda dentro de uma JVM. A JVM é um processo do ponto de vista do sistema operacional. Dentro dela, podem existir múltiplas threads: threads de requisição, garbage collector, pool de worker, threads de agendamento, threads de bibliotecas e drivers.

Quando uma requisição HTTP chega ao backend, ela pode ser tratada por uma thread. Se essa thread faz uma chamada bloqueante ao banco, ela pode ficar bloqueada esperando E/S. Em arquiteturas reativas, tenta-se reduzir bloqueios, mas o sistema operacional continua gerenciando CPU, memória, sockets, arquivos e interrupções.

5.4 Condições de corrida, seção crítica e sincronização

Condição de corrida ocorre quando o resultado depende da ordem de execução de threads/processos concorrentes. Se duas threads atualizam a mesma variável sem controle, o resultado pode ser inconsistente. **Seção crítica é trecho de código que acessa recurso compartilhado e precisa de controle.** Mutex, semáforo, monitor e locks são mecanismos de sincronização.

Pegadinha FGV: thread facilita concorrência, mas não garante automaticamente segurança. Compartilhar memória é vantagem de desempenho e comunicação, mas também fonte de erro concorrente.

6. Escalonamento de CPU

Escalonamento é a **função do sistema operacional que escolhe qual processo ou thread pronta usará a CPU**. Em sistemas multitarefa, essa decisão é contínua. O escalonador busca equilíbrio entre justiça, responsividade, throughput e aproveitamento do processador.

6.1 Preemptivo x não preemptivo

Tipo	Descrição	Exemplo de prova
Não preemptivo	Processo em execução mantém a CPU até bloquear, terminar ou liberar voluntariamente	FCFS simples; processo longo pode prejudicar os demais.
Preemptivo	SO pode interromper processo em execução e passar CPU a outro	Round Robin com quantum; melhora responsividade em sistemas interativos.

Preempção é essencial para sistemas interativos modernos, mas aumenta custo de troca de contexto e exige controle mais cuidadoso.

6.2 Métricas de escalonamento

Métrica	O que mede	Objetivo típico
Utilização da CPU	Percentual de tempo em que CPU executa trabalho útil	Maximizar
Throughput	Quantidade de processos concluídos por unidade de tempo	Maximizar
Turnaround	Tempo total entre submissão e conclusão	Minimizar
Tempo de espera	Tempo em fila de pronto	Minimizar
Tempo de resposta	Tempo até a primeira resposta ao usuário	Minimizar
Justiça	Evitar que processos fiquem indefinidamente sem CPU	Maximizar

6.3 Algoritmos clássicos

Algoritmo	Ideia	Vantagem	Risco/pegadinha
FCFS - First-Come, First-Served	Atende por ordem de chegada	Simples	Processo longo na frente causa efeito comboio.
SJF - Shortest Job First	Menor duração primeiro	Pode reduzir espera média	Precisa estimar duração; pode causar starvation de jobs longos.
SRTF - Shortest Remaining Time First	Menor tempo restante, com preempção	Favorece jobs curtos	Mais trocas e risco de starvation.
Round Robin	Cada processo recebe quantum	Bom para sistemas interativos	Quantum muito pequeno gera overhead; muito grande vira FCFS.
Prioridade	Maior prioridade executa antes	Permite políticas de importância	Baixa prioridade pode sofrer starvation; aging reduz isso.
Filas multinível	Filas separadas por classe de processo	Diferencia interativo, batch, sistema	Configuração pode privilegiar demais uma classe.

6.4 Starvation e aging

Starvation é inanição: um processo fica pronto, mas quase nunca recebe CPU porque outros são sempre preferidos. **Aging** é técnica de envelhecimento: aumenta gradualmente a prioridade de processos que esperam muito, reduzindo o risco de starvation.

FGV costuma explorar troca de objetivo: Round Robin não busca necessariamente menor turnaround médio; ele é lembrado por justiça/fatia de tempo e boa resposta interativa. SJF reduz espera média em condições ideais, mas pode sacrificar processos longos.

7. Gerenciamento de memória

Gerenciamento de memória é o conjunto de mecanismos pelos quais o SO controla quais processos usam quais áreas de memória, com proteção, eficiência e abstração. O processo enxerga endereços lógicos ou virtuais; o hardware e o SO mapeiam isso para memória física.

7.1 Endereço lógico, virtual e físico

Conceito	Descrição	Ponto de prova
Endereço lógico/virtual	Endereço gerado pelo programa/processo	Não é necessariamente o endereço real na RAM.
Endereço físico	Local efetivo na memória principal	Resultado da tradução feita com apoio da MMU.
Espaço de endereçamento	Conjunto de endereços que um processo pode usar	Isola processos e protege memória.

Conceito	Descrição	Ponto de prova
MMU	Unidade de Gerenciamento de Memória	Traduz endereços e aplica proteção.

7.2 Memória virtual

Memória virtual **permite que cada processo enxergue um espaço de endereçamento próprio, independente da disposição física real na RAM.** Ela facilita isolamento, proteção, compartilhamento controlado e execução de programas cujo espaço virtual excede a memória física disponível, usando paginação e, quando necessário, área de troca em disco.

Pegadinha: **memória virtual não torna o disco tão rápido quanto RAM nem elimina necessidade de memória física. Quando o sistema passa a mover páginas demais entre RAM e disco, o desempenho desaba.**

7.3 Paginação

Paginação divide o espaço virtual em páginas de tamanho fixo e a memória física em quadros de mesmo tamanho. Uma tabela de páginas mapeia página virtual para quadro físico. Como páginas e quadros têm tamanho fixo, a paginação evita fragmentação externa, mas pode gerar fragmentação interna na última página alocada.

Termo	Função
Página	Bloco de tamanho fixo no espaço virtual.
Quadro/frame	Bloco de tamanho fixo na memória física.
Tabela de páginas	Estrutura que mapeia página virtual para quadro físico.
TLB	Cache de traduções de páginas usado para acelerar acesso.
Page fault	Falta de página: página necessária não está carregada na RAM.

7.4 Page fault, swapping e thrashing

Page fault ocorre quando o processo acessa uma página que não está na memória física. O SO precisa localizar a página em disco ou em outro local, carregar para um quadro livre ou substituir uma página existente e retomar o processo. Isso é muito mais caro que um acesso normal à RAM.

Swapping é movimentar partes de processos/páginas entre memória e disco. Thrashing ocorre quando o sistema gasta tempo excessivo trocando páginas, com pouca execução útil. Em prova, thrashing costuma vir associado a alta taxa de faltas de página e falta de quadros suficientes.

7.5 Algoritmos de substituição de páginas

Algoritmo	Ideia	Pegadinha
Ótimo	Substitui página que demorará mais para ser usada	Serve como referência teórica; exige conhecer o futuro.
FIFO	Remove a página carregada há mais tempo	Simples, mas pode remover página muito usada.
LRU - Least Recently Used	Remove a menos recentemente usada	Boa aproximação, mas tem custo de implementação.
Segunda chance/Clock	FIFO com bit de referência para dar nova chance	Não é LIFO; usa bit de referência.

7.6 Segmentação e fragmentação

Segmentação **divide o espaço do processo em segmentos de tamanho variável**, seguindo a lógica do programa: código, dados, pilha, heap, bibliotecas. Pode oferecer proteção por segmento, mas tende a lidar com fragmentação externa. Paginação usa blocos fixos e evita fragmentação externa, mas pode ter fragmentação interna.

Conceito	Onde aparece	Definição curta
Fragmentação interna	Blocos fixos	Sobra dentro do bloco alocado.
Fragmentação externa	Blocos variáveis	Espaços livres espalhados, não contíguos.

8. Entrada/Saída - E/S

Entrada/Saída é a comunicação entre o sistema computacional e dispositivos: disco, teclado, mouse, impressora, placa de rede, SSD, monitor, sensores e outros. O SO abstrai diferenças entre dispositivos, controla acesso concorrente e oferece interfaces padronizadas.

8.1 Dispositivo, controlador e driver

Elemento	Função
Dispositivo	Hardware que realiza entrada, saída ou armazenamento.
Controlador	Componente que controla operação do dispositivo e conversa com barramento/sistema.
Driver	Software do SO que sabe comandar o dispositivo/controlador.
Interrupção	Sinal usado para avisar a CPU de que um evento ocorreu.

8.2 Polling, interrupção e DMA

Método	Como funciona	Vantagem	Cuidado de prova
Polling	CPU consulta repetidamente o dispositivo	Simples	Pode desperdiçar CPU esperando evento.
Interrupção	Dispositivo avisa a CPU quando precisa de atenção	Evita espera ativa contínua	CPU para o fluxo atual para tratar evento.
DMA - Direct Memory Access	Controlador transfere dados diretamente entre dispositivo e memória	Reduz trabalho da CPU em grandes transferências	DMA não significa Direct Access Memory; o correto é Direct Memory Access.

Esse é um ponto com boa chance de cobrança: a descrição “controlador transfere diretamente com a memória e sinaliza ao processador ao final” aponta para DMA.

8.3 Buffer, cache e spool

Técnica	Ideia	Exemplo
Buffer	Área temporária para acomodar diferenças de ritmo entre produtor e consumidor	Buffer de rede ou de leitura de arquivo.
Cache	Cópia de dados frequentemente usados para acelerar acesso futuro	Cache de disco, cache de página, cache HTTP.
Spool	Fila intermediária para dispositivo que atende um trabalho por vez	Fila de impressão.

8.4 E/S bloqueante, não bloqueante, síncrona e assíncrona

Uma chamada bloqueante faz a thread esperar a conclusão da operação. Uma chamada não bloqueante retorna sem aguardar conclusão imediata. A E/S assíncrona permite que a operação prossiga e a aplicação seja notificada depois. Em APIs corporativas, isso aparece em chamadas HTTP, banco de dados, filas, leitura de arquivo e sockets.

Cuidado: não confunda E/S assíncrona com paralelismo. Uma aplicação pode iniciar operação assíncrona e continuar trabalhando, mas o hardware/SO/event loop/pool de threads é que coordenam a conclusão.

9. Sistemas de arquivos

Sistema de arquivos é o componente do SO que organiza dados persistentes em unidades como arquivos e diretórios, mantendo nomes, caminhos, permissões, metadados e mapeamento para blocos físicos ou lógicos de armazenamento.

9.1 Arquivos, diretórios e metadados

Conceito	Descrição
Arquivo	Conjunto nomeado de dados persistentes.
Diretório	Estrutura que organiza nomes e referências a arquivos/outras diretórios.
Caminho absoluto	Identifica o arquivo a partir da raiz do sistema de arquivos.
Caminho relativo	Identifica a partir do diretório atual.
Metadados	Informações sobre arquivo: tamanho, dono, permissões, timestamps, localização, tipo.
Descritor de arquivo	Identificador usado por processo para referenciar arquivo aberto.

9.2 Permissões e proteção

Sistemas de arquivos controlam quem pode ler, escrever ou executar arquivos. Em ambientes multiusuário, isso é essencial para confidencialidade, integridade e disponibilidade. Em Unix/Linux, a lógica clássica separa permissões de usuário dono, grupo e outros; em Windows, é comum falar em ACLs - Access Control Lists.

9.3 Alocação e journaling

Tema	Ideia de prova
Alocação contígua	Arquivo ocupa blocos consecutivos; bom desempenho sequencial, mas sofre com crescimento e fragmentação.
Alocação encadeada	Blocos ligados por ponteiros; evita contiguidade, mas dificulta acesso aleatório.
Alocação indexada	Estrutura de índice aponta para blocos do arquivo; melhora acesso e flexibilidade.
Journaling	Registra operações/metadados antes de aplicá-las, ajudando recuperação após falha.

9.4 Conexão com dev/backend

Quando sua aplicação salva logs, lê um arquivo de configuração, gera um relatório PDF, grava upload ou abre socket, há recursos gerenciados pelo sistema operacional. Se a aplicação esquece de fechar descritores, consome memória demais ou cria threads em excesso, o problema deixa de ser apenas de código e vira problema de recurso operacional.

10. Pegadinhas FGV da aula

Pegadinha	Correção para prova
Programa e processo são sinônimos.	Programa é código/arquivo; processo é instância em execução com estado.
Processo pronto está esperando E/S.	Pronto espera CPU; bloqueado espera evento/recurso.
Escalonamento é a mesma coisa que troca de contexto.	Escalonamento decide; troca de contexto executa a troca salvando/restaurando estado.
Thread sempre é mais segura que processo.	Thread é mais leve, mas compartilha memória e pode gerar corrida.
Concorrência é sempre paralelismo.	Concorrência pode ocorrer por alternância em um único núcleo; paralelismo exige execução simultânea real.

Pegadinha	Correção para prova
Round Robin minimiza sempre o turnaround.	Round Robin é associado a quantum, justiça e responsividade, não a menor turnaround em todo cenário.
Memória virtual é RAM extra.	É abstração/mecanismo de endereçamento; pode usar disco, com custo alto.
Paginação elimina toda fragmentação.	Elimina fragmentação externa, mas pode gerar fragmentação interna.
Segmentação e paginação são a mesma coisa.	Segmentação usa unidades lógicas variáveis; paginação usa blocos fixos.
DMA significa Direct Access Memory.	O correto é Direct Memory Access.
Cache, buffer e spool são equivalentes.	Cache acelera reuso; buffer acomoda ritmos; spool enfileira trabalho para dispositivo.
Sistema de arquivos é só interface gráfica de pastas.	É estrutura lógica de persistência, nomes, metadados, permissões e mapeamento de blocos.

11. Exemplos aplicados ao seu contexto

11.1 Backend Java/Quarkus em produção

Imagine uma API Quarkus em um servidor Linux. Para o SO, a JVM é um processo. Dentro dela, há várias threads. Uma thread atende uma requisição HTTP, valida JWT, consulta banco, serializa JSON e responde. Quando a thread espera o banco, pode ficar bloqueada em E/S. O escalonador decide quais threads executam. O gerenciador de memória traduz endereços virtuais em físicos. O sistema de arquivos armazena logs, certificados, configurações e artefatos. A rede usa sockets, mas socket também é recurso controlado pelo SO.

11.2 Quando o problema de aplicação vira problema de sistema operacional

Sintoma	Possível relação com SO
API lenta sob carga	Threads bloqueadas em E/S, CPU saturada, escalonamento, locks ou GC.
Servidor com uso alto de swap	Pressão de memória, paginação excessiva, risco de thrashing.
Muitos erros de “too many open files”	Descritores de arquivo/sockets não fechados ou limite operacional baixo.
Logs demorando ou disco em 100%	Gargalo de E/S, flush síncrono, filesystem, storage ou fila de escrita.
Aplicação responsiva mas CPU alta	Carga CPU-bound, pool de threads, loops, criptografia, serialização pesada.
Aplicação travada esperando retorno externo	Chamada bloqueante de rede/banco/fila sem timeout adequado.

12. Lista de questões autorais - padrão FGV

Instruções: resolva as 20 questões sem consultar o gabarito. As questões misturam conceito direto, cenário prático, afirmativas, relação de colunas e interpretação de situação técnica. Há uma única alternativa correta em cada item.

Questão 1

Em um servidor de aplicação, uma instância da JVM executa uma aplicação Java/Quarkus. Em determinado momento, a aplicação possui várias threads atendendo requisições HTTP. Do ponto de vista do sistema operacional, a formulação mais adequada é:

- (A) a JVM é uma thread do sistema operacional, e cada requisição HTTP cria necessariamente um processo independente.
- (B) a aplicação Java dispensa processo, pois o bytecode executa diretamente sobre o hardware por meio da JVM.
- (C) a JVM é tipicamente um processo, dentro do qual podem existir múltiplas threads que compartilham recursos do processo.
- (D) cada thread possui um espaço de endereçamento totalmente isolado das demais threads do mesmo processo.
- (E) o sistema operacional não participa da execução de aplicações Java, pois o gerenciamento fica restrito ao garbage collector.

Questão 2

Um processo encontra-se com todos os recursos necessários para executar, mas ainda não recebeu a CPU porque outro processo está em execução. No modelo clássico de estados de processos, ele está no estado:

- (A) bloqueado, pois aguarda a conclusão de uma operação de entrada/saída.
- (B) terminado, pois ainda não recebeu tempo de processador.
- (C) novo, pois o processo ainda não foi carregado na memória.
- (D) em espera por evento externo, pois qualquer espera no sistema operacional é bloqueio.
- (E) pronto, pois aguarda apenas ser escolhido pelo escalonador para usar a CPU.

Questão 3

Analise as afirmativas sobre processos e threads. I. Threads de um mesmo processo compartilham, em geral, o espaço de endereçamento do processo. II. A criação e a troca de threads tendem a ser mais leves que a criação e a troca de processos. III. Threads eliminam a necessidade de sincronização, pois cada uma possui heap independente. Está correto o que se afirma em:

- (A) I, II e III.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) III, apenas.

Questão 4

Em um sistema interativo, o administrador deseja evitar que um processo CPU-bound monopolize o processador por longos períodos. O algoritmo clássico que utiliza fatias de tempo, denominadas quantum, para alternar a execução entre processos prontos é:

- (A) Round Robin.
- (B) First-Come, First-Served.
- (C) Shortest Job First não preemptivo.
- (D) Alocação contígua.
- (E) Substituição de páginas por FIFO.

Questão 5

Relacione os conceitos da coluna da esquerda com a descrição mais adequada. 1. Escalonamento. 2. Troca de contexto. 3. Interrupção. 4. DMA. () Salva e restaura o estado de execução para alternar a CPU entre entidades executáveis. () Decide qual processo ou thread pronta receberá o processador. () Permite que controlador transfira dados diretamente entre dispositivo e memória. () Sinaliza à CPU a ocorrência de evento que requer tratamento. A sequência correta é:

- (A) 1 - 2 - 3 - 4.
- (B) 2 - 1 - 4 - 3.
- (C) 3 - 4 - 1 - 2.
- (D) 4 - 3 - 2 - 1.
- (E) 2 - 4 - 1 - 3.

Questão 6

Sobre memória virtual, assinale a afirmativa correta.

- (A) Elimina a necessidade de memória RAM física, pois todo acesso passa a ocorrer diretamente no disco.
- (B) Permite a cada processo enxergar um espaço de endereçamento próprio, com tradução para memória física feita com apoio do hardware e do sistema operacional.
- (C) Impede a execução simultânea de processos, pois todos passam a compartilhar a mesma área física sem proteção.
- (D) É sinônimo de memória cache da CPU, pois ambas armazenam traduções de endereços.
- (E) Substitui o sistema de arquivos, pois arquivos passam a ser armazenados no espaço virtual do processo.

Questão 7

Em um sistema com paginação, ocorre uma falta de página quando:

- (A) a CPU executa uma instrução privilegiada em modo kernel.
- (B) um arquivo é aberto por mais de um processo simultaneamente.
- (C) a página virtual referenciada pelo processo não está carregada em memória física no momento do acesso.
- (D) um processo pronto ainda não recebeu processador.
- (E) o escalonador escolhe processo de menor prioridade.

Questão 8

Considere as afirmações sobre fragmentação. I. A fragmentação interna está associada à sobra de espaço dentro de blocos alocados. II. A fragmentação externa ocorre quando espaços livres existem, mas estão espalhados de modo a dificultar alocações contíguas. III. A paginação tende a eliminar fragmentação externa, mas pode gerar fragmentação interna. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

Questão 9

Um servidor apresenta alta taxa de faltas de página, uso intenso de disco e baixa progressão real das aplicações. O sistema passa mais tempo transferindo páginas entre memória e disco do que executando trabalho útil. Esse fenômeno é conhecido como:

- (A) starvation.
- (B) thrashing.
- (C) spooling.
- (D) aging.

(E) polling.

Questão 10

No gerenciamento de entrada/saída, o método em que a CPU consulta repetidamente o estado de um dispositivo para verificar se ele está pronto é denominado:

(A) DMA.

(B) interrupção mascarável.

(C) spooling.

(D) polling.

(E) paginação por demanda.

Questão 11

Em uma operação de entrada/saída de grande volume, deseja-se reduzir a participação da CPU na cópia dos dados. O controlador do dispositivo passa a transferir dados diretamente para a memória principal e avisa ao processador ao término. O mecanismo descrito é:

(A) spool.

(B) cache associativo.

(C) DMA - Direct Memory Access.

(D) Round Robin.

(E) segmentação.

Questão 12

Assinale a opção que diferencia corretamente buffer, cache e spool.

(A) Buffer acomoda diferenças de ritmo; cache acelera reuso de dados; spool organiza fila intermediária para dispositivo ou serviço.

(B) Buffer é memória virtual; cache é fila de impressão; spool é tabela de páginas.

(C) Buffer, cache e spool são nomes equivalentes para a mesma técnica de alocação de memória.

(D) Cache só existe em navegadores; buffer só existe em bancos de dados; spool só existe em redes.

(E) Spool é usado para substituir o escalonador de processos em sistemas preemptivos.

Questão 13

Um sistema de arquivos mantém informações como dono, permissões, tamanho, datas de criação/modificação e localização lógica dos blocos de um arquivo. Essas informações são denominadas:

(A) quantum.

(B) TLB.

(C) seção crítica.

(D) page fault.

(E) metadados.

Questão 14

Em relação a algoritmos de escalonamento, assinale a afirmativa correta.

(A) FCFS é preemptivo por definição e sempre oferece o menor tempo de resposta em sistemas interativos.

(B) SJF independe de estimativa de duração dos processos e não apresenta risco de starvation.

(C) Round Robin utiliza quantum; quantum muito pequeno pode aumentar overhead de troca de contexto.

(D) Escalonamento por prioridade impede starvation mesmo quando prioridades nunca são ajustadas.

(E) SRTF é a versão não preemptiva do algoritmo de prioridade fixa.

Questão 15

Considere um processo que tenta ler dados de disco. Enquanto a operação de E/S não termina, ele deixa de disputar a CPU e aguarda o evento de conclusão. Nesse intervalo, o estado clássico mais adequado para o processo é:

- (A) pronto.
- (B) novo.
- (C) em execução.
- (D) bloqueado ou em espera.
- (E) terminado.

Questão 16

Sobre modo usuário, modo kernel e chamadas de sistema, assinale a alternativa correta.

- (A) Aplicações em modo usuário executam com os mesmos privilégios do kernel para reduzir overhead.
- (B) Chamadas de sistema permitem que aplicações acessem serviços do kernel de forma controlada.
- (C) Drivers de dispositivo são executados exclusivamente como bibliotecas Java em modo usuário.
- (D) Modo kernel é usado apenas em sistemas operacionais sem memória virtual.
- (E) O uso de chamadas de sistema elimina a necessidade de permissões em arquivos.

Questão 17

Em sistemas com escalonamento por prioridade, uma técnica que aumenta gradualmente a prioridade de processos que esperam por muito tempo, reduzindo o risco de inanição, é chamada de:

- (A) spooling.
- (B) paginação.
- (C) polling.
- (D) aging.
- (E) journaling.

Questão 18

Assinale a alternativa que melhor descreve a função de um descritor de arquivo em sistemas operacionais.

- (A) É o bloco físico que contém todos os dados do arquivo no disco.
- (B) É o algoritmo que escolhe a próxima página a ser removida da memória.
- (C) É uma referência mantida pelo processo para acessar um arquivo ou recurso aberto, como socket ou pipe.
- (D) É a senha criptografada associada ao arquivo.
- (E) É uma thread especial usada para executar o garbage collector.

Questão 19

Em uma API Java com várias threads acessando uma estrutura compartilhada em memória, o resultado passa a variar conforme a ordem de execução das threads. O problema descrito é mais bem classificado como:

- (A) condição de corrida.
- (B) fragmentação externa.
- (C) page fault.
- (D) journaling.

(E) spooling.

Questão 20

Sobre journaling em sistemas de arquivos, é correto afirmar que:

(A) substitui a memória virtual e impede faltas de página.

(B) elimina a necessidade de permissões de acesso.

(C) registra operações ou metadados antes de sua efetivação, auxiliando a recuperação de consistência após falhas.

(D) é um algoritmo de escalonamento preemptivo baseado em quantum.

(E) é uma técnica de transferência direta entre dispositivo e memória principal.

13. Gabarito resumido

1-C	2-E	3-B	4-A	5-B
6-B	7-C	8-E	9-B	10-D
11-C	12-A	13-E	14-C	15-D
16-B	17-D	18-C	19-A	20-C

14. Comentários completos

Questão 1 - Gabarito: C

Por que a correta está correta: A JVM é vista pelo SO como um processo comum, com espaço de endereçamento e recursos próprios. Dentro dela, há múltiplas threads, inclusive threads de requisição, GC e workers.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Inverte os níveis: uma JVM não é uma thread; cada requisição não precisa virar processo independente.
- (B) Bytecode não dispensa processo nem SO; a JVM executa como processo e usa serviços do sistema.
- (D) Threads do mesmo processo compartilham espaço de endereçamento; não têm isolamento total como processos distintos.
- (E) Garbage collector gerencia memória da aplicação/JVM, mas não substitui o SO.

Erro provável do candidato: confusão entre processo, thread e concorrência.

Questão 2 - Gabarito: E

Por que a correta está correta: O estado pronto indica que o processo pode executar, mas aguarda a CPU ser atribuída pelo escalonador.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Bloqueado seria espera por evento/E/S, não apenas por CPU.
- (B) Terminado é processo finalizado.
- (C) Novo é criação inicial.
- (D) Nem toda espera é bloqueio; pronto espera CPU.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 3 - Gabarito: B

Por que a correta está correta: I e II são corretas: threads compartilham espaço do processo e tendem a ser mais leves que processos. III é falsa porque threads compartilham heap e exigem sincronização quando acessam dados compartilhados.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Inclui III, que erra ao dizer que threads eliminam sincronização.
- (C) Inclui III e exclui II, invertendo a lógica.
- (D) II é correta, mas III é falsa.
- (E) III é justamente a falsa.

Erro provável do candidato: confusão entre processo, thread e concorrência.

Questão 4 - Gabarito: A

Por que a correta está correta: Round Robin usa quantum, alternando processos prontos por fatias de tempo, sendo clássico em sistemas interativos.

Por que as demais parecem tentadoras, mas estão erradas:

- (B) FCFS segue chegada e pode deixar processo longo monopolizar CPU.
- (C) SJF não preemptivo favorece jobs curtos, mas não usa quantum.
- (D) Alocação contígua é tema de memória/arquivo, não escalonamento.
- (E) FIFO de páginas é substituição de memória, não escalonamento de CPU.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 5 - Gabarito: B

Por que a correta está correta: Troca de contexto salva/restaura; escalonamento decide; DMA transfere dispositivo-memória; interrupção sinaliza evento à CPU.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Inverte escalonamento e troca de contexto e troca DMA/interrupção.
- (C) Coloca interrupção como troca de contexto e DMA como escalonamento.
- (D) Atribui DMA à troca de contexto e interrupção ao escalonamento.
- (E) Acerta a primeira posição, mas troca DMA e escalonamento.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 6 - Gabarito: B

Por que a correta está correta: Memória virtual oferece espaço de endereçamento próprio e protegido, com tradução para memória física pela MMU e pelo SO.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Disco não substitui RAM; usar disco como swap é muito mais lento.
- (C) Memória virtual favorece isolamento e multiprogramação.
- (D) TLB é cache de traduções; memória virtual é conceito mais amplo.
- (E) Sistema de arquivos continua existindo; memória virtual não o substitui.

Erro provável do candidato: lacuna em memória virtual, paginação e fragmentação.

Questão 7 - Gabarito: C

Por que a correta está correta: Page fault ocorre quando a página virtual referenciada não está presente na memória física.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Instrução privilegiada envolve modo kernel/usuário, não falta de página por si.
- (B) Arquivo aberto por múltiplos processos é tema de arquivo/concorrência.
- (D) Processo pronto esperando CPU é escalonamento.
- (E) Prioridade é política de escalonamento.

Erro provável do candidato: lacuna em memória virtual, paginação e fragmentação.

Questão 8 - Gabarito: E

Por que a correta está correta: As três afirmações estão corretas: interna é sobra dentro do bloco; externa é espaço livre espalhado; paginação remove externa, mas pode causar interna.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) I está correta, mas incompleta.
- (B) II está correta, mas incompleta.
- (C) Falta III, que é central em paginação.
- (D) Falta I, que define fragmentação interna.

Erro provável do candidato: lacuna em memória virtual, paginação e fragmentação.

Questão 9 - Gabarito: B

Por que a correta está correta: Thrashing é a degradação causada por paginação excessiva, com muitas faltas de página e pouco progresso útil.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Starvation é falta de atendimento de processo pronto, geralmente por prioridade.
- (C) Spooling é enfileiramento para dispositivo/serviço.
- (D) Aging combate starvation.
- (E) Polling é consulta repetida a dispositivo.

Erro provável do candidato: lacuna em memória virtual, paginação e fragmentação.

Questão 10 - Gabarito: D

Por que a correta está correta: Polling é a consulta repetitiva do estado do dispositivo pela CPU.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) DMA reduz participação da CPU em transferência de dados.
- (B) Interrupção é dispositivo avisando a CPU, não consulta ativa.
- (C) Spooling é fila intermediária.
- (E) Paginação por demanda é memória virtual.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 11 - Gabarito: C

Por que a correta está correta: DMA significa Direct Memory Access: transferência direta entre dispositivo/controlador e memória, com menor carga para CPU.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Spool organiza fila de trabalhos.
- (B) Cache associativo acelera acesso por reuso/localidade.
- (D) Round Robin é escalonamento por quantum.
- (E) Segmentação é gerência de memória.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 12 - Gabarito: A

Por que a correta está correta: A alternativa diferencia corretamente as três técnicas: buffer para ritmos, cache para reuso, spool para fila intermediária.

Por que as demais parecem tentadoras, mas estão erradas:

- (B) Mistura conceitos sem relação.
- (C) São técnicas diferentes, embora todas possam usar memória/armazenamento.
- (D) Restringe indevidamente tecnologias que aparecem em vários contextos.
- (E) Spool não substitui escalonador.

Erro provável do candidato: confusão entre técnicas de E/S, arquivos e metadados.

Questão 13 - Gabarito: E

Por que a correta está correta: Metadados são dados sobre o arquivo, como dono, permissões, tamanho, timestamps e localização.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Quantum é fatia de tempo de escalonamento.
- (B) TLB é cache de tradução de endereços.
- (C) Seção crítica é trecho com recurso compartilhado.
- (D) Page fault é falta de página.

Erro provável do candidato: confusão entre técnicas de E/S, arquivos e metadados.

Questão 14 - Gabarito: C

Por que a correta está correta: Round Robin usa quantum, e quantum muito pequeno causa muitas trocas de contexto, aumentando overhead.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) FCFS não é preemptivo por definição e pode ter má resposta.
- (B) SJF depende de estimativa e pode causar starvation.
- (D) Prioridade pode causar starvation se não houver aging.
- (E) SRTF é preemptivo e ligado ao menor tempo restante.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 15 - Gabarito: D

Por que a correta está correta: Enquanto espera E/S, o processo fica bloqueado/em espera; não disputa CPU até o evento.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Pronto espera apenas CPU.
- (B) Novo é criação inicial.
- (C) Em execução significa usando CPU.
- (E) Terminado significa finalizado.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 16 - Gabarito: B

Por que a correta está correta: Chamadas de sistema são a interface controlada pela qual aplicações solicitam serviços ao kernel.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Modo usuário tem privilégios restritos.
- (C) Drivers não são bibliotecas Java por definição; muitos operam em modo kernel.
- (D) Modo kernel é característica de proteção, não depende de ausência de memória virtual.
- (E) Syscalls não eliminam permissões; elas as respeitam/verificam.

Erro provável do candidato: conceito técnico próximo aplicado fora do contexto.

Questão 17 - Gabarito: D

Por que a correta está correta: Aging eleva gradualmente a prioridade de processos esperando há muito tempo, combatendo starvation.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Spooling é fila intermediária.
- (B) Paginação é técnica de memória.
- (C) Polling é consulta repetida de dispositivo.
- (E) Journaling é sistema de arquivos.

Erro provável do candidato: troca entre escalonamento, estado de processo e mecanismos de E/S.

Questão 18 - Gabarito: C

Por que a correta está correta: Descritor de arquivo é uma referência usada por processo para acessar arquivo/recurso aberto.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Não é o bloco físico do arquivo.
- (B) Não é algoritmo de substituição de páginas.
- (D) Não é senha.
- (E) Não é thread de garbage collector.

Erro provável do candidato: confusão entre técnicas de E/S, arquivos e metadados.

Questão 19 - Gabarito: A

Por que a correta está correta: Condição de corrida ocorre quando o resultado depende da ordem de execução concorrente.

Por que as demais parecem tentadoras, mas estão erradas:

- (B) Fragmentação externa envolve alocação de memória/armazenamento em blocos variáveis.
- (C) Page fault é falta de página.
- (D) Journaling é recuperação de sistema de arquivos.
- (E) Spooling é fila para dispositivo/serviço.

Erro provável do candidato: confusão entre processo, thread e concorrência.

Questão 20 - Gabarito: C

Por que a correta está correta: Journaling registra operações ou metadados para permitir recuperação consistente após falhas.

Por que as demais parecem tentadoras, mas estão erradas:

- (A) Não substitui memória virtual.
- (B) Não elimina controle de permissões.
- (D) Não é escalonamento por quantum.
- (E) Essa é descrição de DMA.

Erro provável do candidato: confusão entre técnicas de E/S, arquivos e metadados.

15. Checklist final do que memorizar

- Sistema operacional gerencia CPU, memória, entrada/saída, arquivos, segurança e abstrações para aplicações.
- Kernel executa em modo privilegiado; aplicações executam em modo usuário e usam chamadas de sistema.
- Programa é código; processo é execução com estado e recursos.
- Pronto espera CPU; bloqueado espera evento ou recurso; em execução usa CPU.
- Escalonamento decide quem executa; troca de contexto salva e restaura estado.
- Thread é fluxo de execução dentro de processo; compartilha memória, mas tem pilha/contexto próprios.
- Concorrência não é necessariamente paralelismo.
- Round Robin usa quantum; SJF favorece jobs curtos; prioridade pode causar starvation; aging combate starvation.
- Memória virtual fornece espaço de endereçamento lógico/virtual com tradução para memória física.
- Paginação usa páginas e quadros de tamanho fixo; pode gerar page fault, swapping e thrashing.
- Paginação elimina fragmentação externa, mas pode gerar fragmentação interna.
- Polling consulta repetidamente; interrupção avisa evento; DMA transfere diretamente com a memória.
- Buffer acomoda ritmos; cache acelera reuso; spool enfileira trabalhos.
- Sistema de arquivos organiza arquivos, diretórios, permissões, metadados e blocos.

16. Fontes técnicas e observações

Este material foi produzido com base no edital retificado do TJSC 2026, na trilha deste chat e no Manual Mestre de Calibração FGV do projeto. As questões são autorais e inéditas; provas reais foram usadas apenas para calibrar estilo, densidade e tipo de pegadinha.

Fonte	Uso na aula
Edital TJSC 2026 - retificado em 10/04/2026	Confirma a disciplina Infraestrutura de TI e Computação em Nuvem e o item de sistemas operacionais: processos, memória, entrada e saída.
Manual Mestre de Calibração FGV para Aulas e Questões	Régua editorial: aula aderente, progressiva, sem gabarito artificial, com distratores fortes e comentários completos.
FGV - prova AMAZUL 2026 - Técnico de Informática	Calibração de estilo FGV em memória virtual, threads, OSI, entrada/saída e TI técnica.
Qconcursos - questões FGV e correlatas de Sistemas Operacionais	Mapeamento de termos recorrentes: escalonamento, memória virtual, page fault, DMA, processos e threads.
Linux Kernel Documentation - CFS Scheduler	Referência técnica sobre escalonamento moderno no Linux.
Microsoft Learn - Kernel-Mode Driver Architecture	Referência técnica sobre gerenciadores de processos/threads, memória, I/O e drivers no Windows.

Fonte	Uso na aula
man7.org - pthreads(7)	Referência técnica sobre POSIX threads e compartilhamento de atributos entre threads.
The Open Group POSIX - fork()	Referência de criação de processos em sistemas POSIX.

Links consultados: <https://conhecimento.fgv.br/concursos/tjscservidor26> ;
<https://docs.kernel.org/scheduler/sched-design-CFS.html> ;
<https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/> ;
<https://man7.org/linux/man-pages/man7/pthreads.7.html> ;
<https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html> .

Fim da Aula 6.